

Midas Touch: Invalid-Data Assisted Reliability and Performance Boost for 3D High-Density Flash

Qiao Li*, Hongyang Dang*, Zheng Wan*, Congming Gao*✉
Min Ye†, Jie Zhang‡, Tei-Wei Kuo§, Chun Jason Xue¶

*Xiamen Key Laboratory of Intelligent Storage and Computing, Xiamen University, †City University of Hong Kong

‡Peking University, §National Taiwan University, ¶Mohamed bin Zayed University of Artificial Intelligence

Abstract—High-density 3D NAND flash like QLC (Quad-Level Cell) is prevailing in providing large capacities for data-intensive applications. Because of the structure limitation, a two-step programming with a specific sequence is adopted in 3D QLC flash, where data could become invalid between the two programming steps. This is called *invalid programming*, as the second-step programming is conducted on partially-invalid wordlines (WLs). By exploiting this phenomenon, this work proposes invalid-data assisted strategies for performance and reliability boosting of valid data in 3D QLC-based flash storage systems. We first propose a high-efficiency re-programming (RP) scheme to re-program the valid data and a high-reliability not-programming (NP) scheme to program data on the partially-invalid WLs. An adaptive data allocation (ADA) strategy for data management between the SLC and QLC regions is further introduced to reduce the occurrence of invalid programming. The simulator-based experiments show the proposed RP scheme combined with ADA can reduce the execution time for programming by 13.51%, on average. Through real-device evaluations, we present that the NP scheme can reduce the bit error rate of NP-programmed data by 32.8% of the worst page type, thus improving overall reliability, which translates to 12% reduction in refreshing overheads and 30% lifetime extension, on average. Besides, the NP scheme with ADA averagely reduces energy consumption by 4.8%.

I. INTRODUCTION

With the advantages of low costs and high performance, high-density 3D NAND flash memories are widely deployed as data storage in mobile devices, PC/laptops, and data centers. In recent years, QLC (quad-level cell) flash with four bits per cell has prevailed to provide large capacities for data-intensive applications. Because of the narrow distribution of threshold voltages, the traditional page-granularity based programming is not viable in 3D QLC NAND. Instead, a two-step wordline (WL) based programming strategy is adopted in 3D QLC flash [2], [4], [42]. All four data pages of a WL are simultaneously programmed to the flash cell array in a two-step fashion. Specifically, the first step is coarse programming on the WL, while the second step, which is fine programming, is delayed after the completion of the first step on several other WLs. The programming sequence could lead to the phenomenon that some data become invalid between the two programming steps. This is a new issue, called *invalid programming*, in 3D QLC flash introduced by the new programming strategy, which has not been addressed in the literature. This work exploits this

phenomenon for performance and reliability improvement in 3D QLC flash memories.

The existing research in optimizing high-density 3D flash has proposed strategies from different angles: coding [17], [32], re-programming [18], [20], [30], and hybrid space management [10], [29], [41], [46]. With a large number of voltage states, 3D QLC flash presents great exploration space in the performance and reliability tradeoff. Lv et al. [32] studied the tradeoff in different gray codes and proposed to utilize multiple gray codes in a hybrid fashion for 3D QLC flash. Several studies proposed to re-program TLC/QLC flash cells by first using the low voltage states, then re-programming flash cells to high voltage states [30]. As QLC can only endure a limited number of P/E cycles, re-programming and write-once memory (WOM) code [18] could increase logical writable data before an erase. Due to the limited performance and lifetime, a single-level cell (SLC) region is usually equipped in QLC flash-based solid-state drives (SSDs) to construct a hybrid SSD structure. The SLC region could serve as a buffer for absorbing write requests and relieving the wear-out of QLC region [10], or providing high access performance [29]. In such hybrid SSDs, proper data allocation and management are critical for the performance and lifetime of SSD [41]. While these strategies relieve the weakness of high-density 3D flash, none of them take the *invalid programming* problem into consideration.

In this paper, we propose new data programming strategies to deal with the inefficiency caused by *invalid programming*. The first scheme is valid data re-programming (RP) to improve the programming efficiency. Because the programming time in the second step is much longer than the first step (e.g., $2\times$ in the flash chips evaluated in our work), we propose to skip the second programming step of partially-invalid WLs and re-program the valid pages to other WLs. The rationale is to avoid applying long-latency programming, which is supposed to be undertaken by four pages, for partially-invalid WLs. Though valid pages experience another first-step programming, the average time cost in the second-step programming for each page will be reduced. As re-programming does not introduce benefits when there are too many valid pages (2 or 3 as indicated by evaluations), the second-step programming needs to be performed on these partially-invalid WLs. We then propose a not-programming (NP) scheme for the second-step

programming by exploiting the invalid data for reliability enhancement on the valid data of the partially-invalid WLs. As invalid pages will not be accessed, it is not necessary to provide a reliability guarantee for them. We propose to revise the voltage states during the second programming step by setting enlarged noise margins between the voltage states that correspond to valid pages, for enhancing data reliability for valid pages. Finally, we propose an adaptive data allocation strategy between the SLC and QLC regions to reduce the occurrence of invalid programming. Our evaluation reveals that there exists a small portion of data that introduces invalid programming repeatedly. Based on the observation, we propose to program these data to the SLC region without introducing performance degradation or frequent data migration. The proposed strategies are evaluated on real 3D NAND QLC flash chips and widely used simulator. Benefiting from the skip of time-consuming second-step programming, the proposed RP scheme combined with ADA reduces the execution time for data programming by 13.51%. The average raw bit error rate (RBER) of NP-programmed data is reduced by 32.8%, which directly improves the overall reliability by reducing the 99th RBER by 9.35%. We further propose to separately refresh the normally-programmed WLs (normal refresh interval) and NP-programmed WLs (prolonged refresh interval), as NP-programmed WLs have much lower RBER. The evaluations show a 12% reduction in refreshing overheads and a 30% lifetime extension. In addition, the NP scheme with ADA reduces energy consumption by 4.8%, on average.

The major contributions of this work are as follows.

- We propose a new re-programming scheme to skip the time-consuming second-step programming on partially-invalid WLs, for enhancing the programming performance by reducing the average program latency of each valid page;
- Motivated by the real-chip testing, we propose a not-programming scheme where voltage states corresponding to invalid pages are not programmed in the second-step programming for enhancing the reliability of valid pages;
- We propose a new adaptive data allocation strategy between SLC and QLC regions to reduce the occurrence of invalid programming.

In the remaining of this paper, Section II presents the background. Section III presents the motivation of this work. The proposed strategies are introduced in Section IV and Section V presents the evaluations. Section VI presents the related works. Section VII concludes this work.

II. BACKGROUND

A. 3D QLC Flash Memory

In NAND flash memory, different types of cells are classified based on the number of bits they can store. These types include SLC (single-level cell), MLC (multi-level cell), TLC (triple-level cell), QLC (quad-level cell), and PLC (penta-level cell). In SLC, each cell can store 1 bit of data, while in QLC, each cell can store 4 bits of data. The number of

bits stored is determined by the number of threshold voltage states that flash cells are divided into. For example, in QLC flash memory, a cell can be programmed to 16 different voltage states, with each voltage state precisely representing a combination of 4 bits of data. The four bits in QLC flash memory are differentiated as the least significant bit (LSB), central significant bit (CSB), most significant bit (MSB), and top significant bit (TSB) [32], [39]. As a result, a single WL in QLC flash memory can store data from 4 pages.

In terms of the same number of cells, QLC flash memory offers four times the storage capacity compared to SLC flash memory. However, even with this increased capacity, flash chips are still unable to keep up with the growing demand for data storage. In order to further increase the storage capacity, researchers have adopted a stacking approach to construct three-dimensional (3D) structured flash chips from planar NAND [5], [16]. In 3D architecture, a flash block is divided into multiple layers, with each layer housing multiple WLs. These WLs are positioned adjacently in the structure, and different layers are vertically interconnected through bitlines.

In 3D QLC flash memory, data read is performed by comparing the voltage of the target cell with the read reference voltage to determine whether the data stored in the cell is 1 or 0 [28], [38], [40]. The specific process is illustrated in Figure 1. To read the LSB data in a certain cell, the voltage of the cell is compared with four read reference voltages: V_4 , V_{10} , V_{12} and V_{14} . If the voltage is higher than V_4 but lower than V_{10} , or if the voltage falls within the range of V_{12} to V_{14} , the stored data bit is identified as 0. In other cases, the stored data is identified as 1. In this reading method, most of the read errors occur in cells near the read reference voltage. This is because these cells are more likely to cross the read reference voltage due to voltage shifting, resulting in the misreading of bit data. Error correction codes (ECC) can correct a certain range of raw bit errors, but when the wear level of cells increases or the retention time is prolonged, the error correction capability reaches its limit, leading to the inability of correctly reading the data [15], [21], [52].

B. Two-step Programming in 3D QLC NAND

In 3D QLC NAND, the gray code is employed to map 4-bit data to 16 voltage levels (from S_0 to S_{15}) of flash cells. The distinctive characteristic of this mapping is that neighboring voltage states correspond to data values that differ by only one bit [13], [32]. As depicted in Figure 1, the voltage states S_5 and S_6 are adjacent, while their stored data values of 0010 and 1010 differ by only 1 bit.

For 3D QLC flash, the programming operation is conducted on a per-WL basis, or more precisely, four pages are simultaneously programmed into the same WL. The voltage state for each cell is determined by the data of four pages to be programmed. To accurately program each cell to its target voltage state, a two-step programming method is adopted [8], [45]. As shown in Figure 2, in the first programming step, all cells of $WL_{i,j}$ (the j_{th} WL of the i_{th} layer) are roughly programmed to a voltage level that is lower than the target

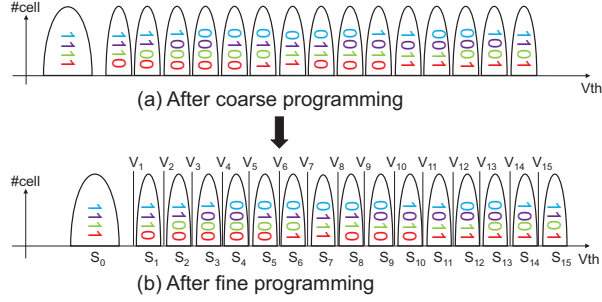


Fig. 1: The threshold voltage distribution after the first and the second programming steps in 3D QLC.

voltage state (Figure 1a). In the second programming step, all cells of $WL_{i,j}$ are programmed to the target voltage states (Figure 1b). It is worth noting that the second-step programming is not performed immediately in order to reduce programming interference and the impact of early charge loss. Hence these two programming steps exhibit loose coupling, and the second-step programming of $WL_{i,j}$ must wait until $WL_{i+1,j}$ completes its coarse programming. For example, the first-step (coarse) programming of $WL_{1,2}$ is performed at step ⑨ in Figure 2, and its second-step (fine) programming is performed at step ⑮. Between the two programming steps of $WL_{1,2}$, several programming steps are performed on other WLs. Thus, there is a long time period between the two programming steps. During this period, the four pages in the buffer may be invalidated by subsequent write requests. As shown in Figure 2, before the second-step programming of $WL_{1,2}$, one of these pages has already become invalid. If the second-step programming still performs normal programming operations on it, it will be a waste of time and energy. This work focuses on turning this waste into gold.

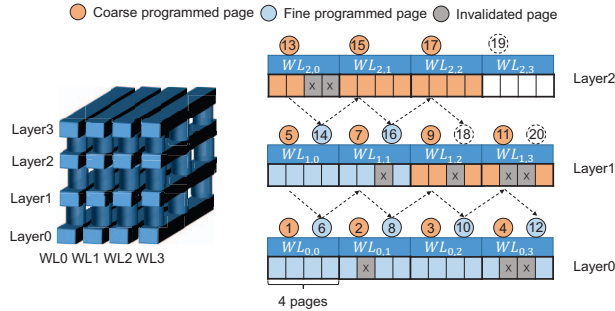


Fig. 2: The 3D NAND structure and the programming sequence of 3D QLC.

C. QLC-based SSD Structure

The SLC-QLC hybrid architecture is often adopted in commercial QLC products, like QLC-based SSD as shown in Figure 3. Due to the higher read/write performance and greater program/erase cycles of SLC NAND, it is often used as a cache for QLC flash, which has relatively lower read/write performance and limited program/erase cycles. The physical

space of QLC-based SSD is divided into SLC region and QLC region, either in terms of hardware or software. The SLC region is utilized for storing hot data, while the QLC region is used for storing cold data. This approach can greatly improve the performance and extend the lifespan of QLC NAND flash.

Since the two-step programming method is adopted for the QLC region, the programming-related information of data pages is more complex, rendering the original framework no longer sufficient to meet new requirements. A program stage table (PST) is introduced to maintain programming-related information for pages being programmed or waiting to be programmed, which also abstracts away the differences in programming methods between SLC and QLC.

For the QLC-based SSD, the write process is illustrated in Figure 3. When a write request arrives at SSD, the data to be written is processed by the host interface logic and written to the buffer, and subsequently written back to the underlying flash array. Before the data is written back, a hot/cold separator is utilized to categorize the data. Hot data is written to SLC regions, while cold data is allocated to QLC regions. Upon the arrival of a write request at the underlying SLC and QLC regions, the controller first acquires one (for SLC region) or four (for QLC region) free pages, then updates the page validity table and mapping table, and subsequently performs data programming.

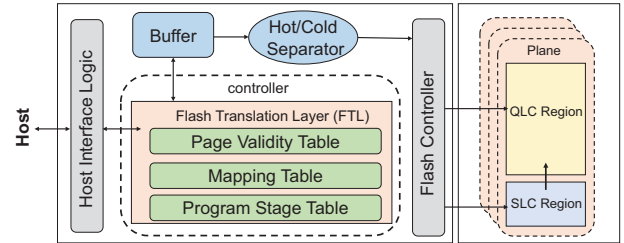
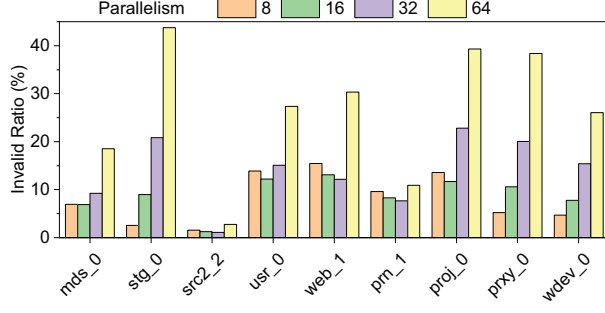


Fig. 3: The QLC-based SSD structure.

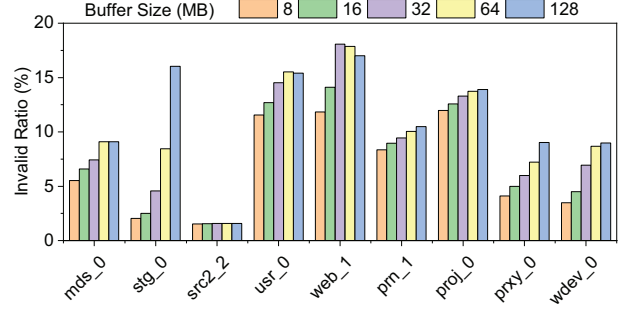
III. MOTIVATION

The *invalid programming* issue refers to the phenomenon of programming invalid pages in the second programming step of 3D QLC flash, which is caused by the loose coupling of the two programming steps in QLC [8]. As shown in Figure 2, the two programming steps are performed alternately, where $WL_{i+1,j}$ completes the first-step programming before the upper-layer $WL_{i,j}$ completes the second-step programming. The programming sequence results in data becoming invalid between two programming steps of a WL.

The *invalid ratio* is defined as the ratio of invalidated page count between two programming steps to the total page count written back from the buffer to the flash array. We have evaluated the invalid ratio of different workloads with different degrees of parallelism and different buffer sizes. The degree of parallelism is represented as the number of open blocks that serve write requests at the same time. The evaluation setting is introduced in Section V and the default setting is 8 open blocks and 16MB buffer size. With more open blocks,



(a) Different degrees of parallelism.



(b) Different buffer sizes.

Fig. 4: The invalid ratio under different degrees of parallelism and different buffer sizes.

data writes can be processed at a higher parallelism degree. Figure 4a shows the invalid ratios with different configurations. In general, the invalid ratio becomes higher with a higher degree of parallelism, because more open blocks serve writes in parallel and the time between the two programming steps becomes longer. Even in the lowest parallelism degree, the ratio can reach 15.4%, which means 15 out of 100 pages that are written back to flash storage trigger invalid programming. We also investigate the impacts of buffer sizes on the invalid ratio under different buffer sizes as shown in Figure 4b. In general, the invalid ratio increases with the increase of the buffer size. This is because a larger buffer can accommodate more write requests, leading to fewer data being written back to the flash arrays. The time period between two programming steps becomes longer and coarse-programmed WLs have a higher probability of becoming partially invalid.

To understand the characteristics of invalid programming, we collect the statistics of logical page numbers (LPNs) that become invalid between two programming steps, as shown in Figure 5. The results indicate that among the pages triggering invalid programming, most LPNs repeatedly encounter invalid programming. Some LPNs might even trigger invalid programming over 100 times.

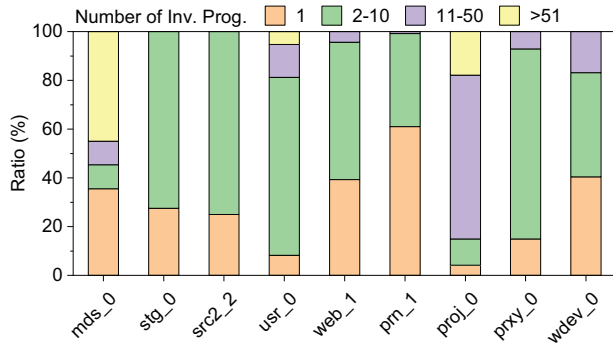


Fig. 5: The breakdown of invalid programming in the occurrence frequency of LPNs.

Based on the results, there is significant potential to reduce the incidence of invalid programming by avoiding re-occurrences of invalid programming on LPNs that have already

experienced it. One simple method is to record all LPNs that have experienced invalid programming and remap their requests to the SLC region (§IV-D). However, even if the data that will be updated are initially written to the SLC region and then migrated to the QLC region, it still cannot completely eliminate the occurrence of invalid programming. Programming invalid data in the second-step programming wastes time and energy. Therefore, it is necessary to design new programming strategies for these partially-invalid WLs (§IV-B and IV-C).

IV. INVALID-DATA ASSISTED PROGRAMMING

A. Overview

The proposed design comprises three components: re-programming module, not-programming module, and adaptive allocator, as shown in Figure 6. Upon the second programming step of a partially-invalid WL, the number of valid pages in the WL is checked. If the valid page count is smaller than a threshold, the re-programming module is applied, which programs valid pages to other free WLs, to skip the time-consuming second-step programming. Otherwise, the not-programming module is used to re-code the data for second-step programming. As the misreading of invalid pages does not affect the data reliability, the not-programming module only programs valid pages in the second-step programming, which allows a broader voltage distribution. To avoid hardware costs, the not-program module only involves data mapping before the second-step programming. Upon data update in QLC, the adaptive data allocator is introduced to be aware of the occurrence of invalid programming events and alleviates the penalty caused by invalid programming via dynamically adjusting the data distribution between SLC and QLC regions.

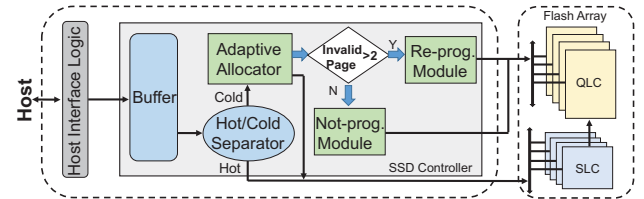


Fig. 6: The architecture of the proposed design.

B. Re-Programming Scheme

In this subsection, we introduce a re-programming (RP) scheme to skip the time-consuming second-step programming on valid pages of some partially-invalid WLs. The re-programming operation is performed by organizing these valid data with other incoming data and re-programming them to other free WLs. Though there is another first-step programming introduced for the re-programmed valid pages, the time-consuming second-step programming is eliminated, thereby improving the overall programming performance. The details about performing the valid page re-programming will be introduced in the following.

Program stage update. The programming stage information is maintained to differentiate the status of data pages in the buffer for programming. According to the process of two-step programming, the page stages are categorized into four types: ‘waiting’ stage, ‘coarse-programmed’ stage, ‘fine-programmed’ stage, and ‘invalid’ stage.

Figure 7 shows an example of re-programming, where three pages (*LPN-5*, *LPN-6* and *LPN-8*) of a WL are invalidated before second-step programming. Considering that the second-step programming incurs a high time cost, which is undertaken by only one valid page *LPN-7*, performing fine programming is inefficient. Instead, re-programming *LPN-7* to other WLs is more efficient. Since the programming stage of *LPN-7* has changed from ‘waiting’ to ‘coarse-programmed’ after coarse programming, it is necessary to update the programming stage back to the ‘waiting’ stage in order to re-program *LPN-7* into other WLs.

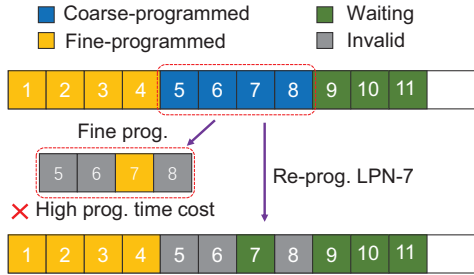


Fig. 7: The illustration of the re-program scheme. *LPN-5*, *LPN-6* and *LPN-8* are invalidated before second-step programming.

Page mapping table maintenance. The page validity bitmap records the validity status of pages. To re-program valid pages into other WLs, it is necessary to mark the valid bits of those pages in the validity bitmap as false, despite that their current valid status is true. Simultaneously, the page mapping table is modified to establish a remapping of LPN to newly allocated physical page numbers (PPN). Throughout this process, utmost consideration must be given to crash consistency [14], [53]. The remap operation must ensure that the valid pages can still be retrieved in the event of a system crash, before its completion. Consequently, the steps involving the modification of the page validity bitmap and page mapping table should be meticulously managed.

Taking Figure 8 as an example, assume that one page, *LPN-7*, needs to be re-programmed. First, by looking up the mapping table, the physical page *blk0-114* corresponding to *LPN-7* is found. Next, we set the valid bit of the corresponding physical page *blk0-114* to 0 in the page validity bitmap. After updating the page validity bitmap, we need to reset the mapping between *LPN-7* and *blk0-114* in the mapping table. After completing these steps, if a system crash were to happen before updating the new mapping entry and validity information, the original information would remain intact and stored in flash memory. This ensures that the valid page can still be read out without compromising data consistency. In the event of a system crash occurring during the re-programming of a valid page, a recovery process is initiated to reconstruct the mapping table and page validity bitmap. This is accomplished by reading the physical-to-logical (P2L) information from the out-of-band area on the physical page.

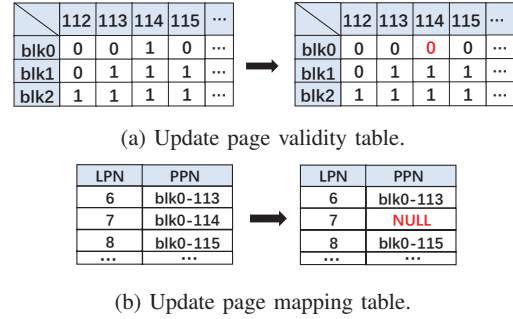


Fig. 8: The process of updating the page mapping table.

Benefits analysis. We denote the time taken by the first-step and second-step programming as T_1 and T_2 , respectively. If all four pages are valid in the second-step programming, the average programming time for each page T_{page} is $(T_1 + T_2)/4$. Suppose there are k ($1 \leq k \leq 3$) valid pages in a partially-invalid WL, the average page programming time T_{page} becomes $(T_1 + T_2)/k$, for these k pages. While in the RP scheme, the average time cost for each of these k pages is $(2T_1 + T_2)/4$, since they will involve two first-step programming and one second-step programming. As presented in Section V, T_2 is about twice T_1 in the evaluated QLC flash chips. Therefore, when k equals to 1 or 2, $(2T_1 + T_2)/4$ is smaller than $(T_1 + T_2)/k$. It means when there are 1 or 2 valid pages on the partially-invalid WLs, we could achieve some benefits in programming time on these valid pages by re-programming them to other WLs. Note that we still introduce extra programming operations to flash storage, which could degrade the overall performance. Aggressively re-programming too many valid pages into free WLs would result in a substantial increase in garbage collection (GC), ultimately leading to poor performance and reduced lifetime of SSDs. The performance impacts are studied in Section V, which shows that re-programming can improve the performance only when there is 1 valid page. For the cases of 2 and 3 valid pages, we still perform the second-step programming on the partially-invalid WLs, where we will explore the invalid data

for reliability improvement as introduced in the following.

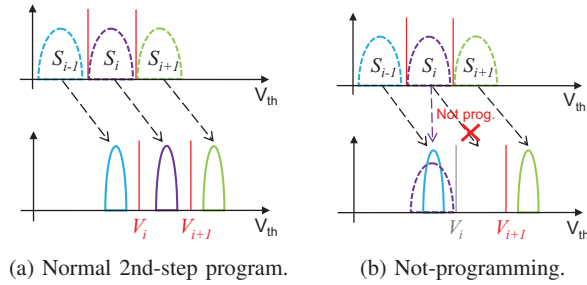


Fig. 9: The not-programming scheme.

C. Not-Programming Scheme

In this subsection, we propose a not-programming (NP) scheme, which inhibits the second-step programming on invalid pages of partially-invalid WLs. The key idea of the NP scheme is to leverage invalid data to improve the reliability of valid data sharing the same WL. Because invalid pages will not be accessed, it is not necessary to provide the reliability guarantee of the voltage states corresponding to invalid pages. In return, the reliability guarantee of invalid pages is rewarded to valid pages by reducing the noise margin between voltage states corresponding to invalid pages but enlarging that corresponding to valid pages.

Figure 9 shows the not-programming strategy, where three adjacent voltage states are presented to illustrate the basic idea. After the first-step programming, flash cells are coarsely programmed to a voltage level that is close to the destination voltage values. The second-step programming will finely program flash cells to the desired voltage states, as shown in Figure 9a. Suppose one page becomes invalid before the second-step programming and V_i is one of the read voltages required to read data from this page. In this case, it is not necessary to differentiate voltage states S_{i-1} and S_i . That is, for state S_i , its programming can be inhibited as V_i will not be applied to differentiate it from state S_{i-1} in future read operations. But for state S_{i-1} , it still has to be finely programmed to higher voltage so as to maintain a large noise margin between S_{i-2} and S_{i-1} . Following that, in the second-step programming, the NP scheme will not program the cells in state S_i . Since the state S_{i+1} is also finely programmed in the second-step programming, the noise margin between the final S_i and S_{i+1} is significantly enlarged as well, which improves the reliability of data pages related to read voltage V_{i+1} .

To verify the feasibility of the NP scheme, we implement the NP scheme on two 3D QLC flash models from different vendors, expressed as Flash A and Flash B. Each block has 96 and 128 layers in Flash A and Flash B, respectively. Figure 10 shows the comparison of the threshold voltage distribution of normal and NP programming. The V_{th} distributions with the normal QLC two-step programming are shown in Figures 10a and 10b, where the dashed lines represent the V_{th} distribution after coarse programming, while the solid lines represent the

V_{th} distribution after fine programming. Figure 10c shows the V_{th} distribution obtained using the NP scheme when the LSB page is invalid for Flash A. This implies that the data from the LSB page will not be read, and therefore, there is no need to apply the four read reference voltages associated with the LSB page, i.e., V_4 , V_{10} , V_{12} , and V_{14} . The two neighboring states corresponding to each of these read voltages overlap with each other, which leaves a larger margin for the neighboring states related to valid pages compared to the distribution presented in Figure 10a. Similarly, when there are two invalid pages, e.g., LSB and CSB pages depicted in Figure 10e, more neighboring voltage states could overlap with each other, while not affecting the reliability of other voltage states corresponding to valid pages. Flash B (Figures 10d and 10f) has a different gray code from Flash A, thus the read voltages required for each page are different and the distributions after the first-step programming are slightly different. We can draw the same conclusion on Flash B that NP programming results in voltage distributions that are friendly to the valid pages on WLs. Therefore, the V_{th} distributions in Figure 10 validate the feasibility of the proposed NP scheme. In the experiment section, the results of RBER reduction on valid pages are collected as the metric to quantitatively analyze the benefits of NP scheme.

To implement the NP scheme, instead of modifying the hardware of the program engine to support NP, we propose a simple coding method, which re-codes the voltage state representation before the second-step programming such that the NP scheme is implemented without any hardware modification. The key idea of the proposed coding method is to remap the bits from S_i to S_{i-1} if the bit that is different between these two states becomes invalid. Figure 11 elaborates this process where the LSB page is set to invalid before the second-step programming. In the absence of LSB data, the valid bits (boxed in this figure) adjacent to the read reference voltages required by reading the LSB page (i.e., V_4 , V_{10} , V_{12} and V_{14} in the figure) become the same. Consequently, while performing NP, we alter the data initially associated with S_i to the data associated with S_{i-1} so that there are no second-step programming targeting on the S_i . Take S_3 and S_4 as an example, where the voltage state is presented as four bits associating with LSB, CSB, MSB, and TSB, respectively. Before the second-step programming, if the LSB of the value 0000 for S_4 is invalidated, the value to be finely programmed is changed to 1000 (S_3). As such, during the execution of the second-step programming, all values associated with S_3 , including the changed and the original values, should be finely programmed to the desired state. But given that the cell voltage of the changed value (i.e., originally programmed to S_4) has approached the desired state (S_3) after the first-step programming, there is no need for additional charge injection into the cell. That is, the programming on invalid pages is inhibited.

For the case of 1 page being invalidated, only two consecutive voltage states have the same data bits on the valid pages. The above strategy could introduce reliability benefits

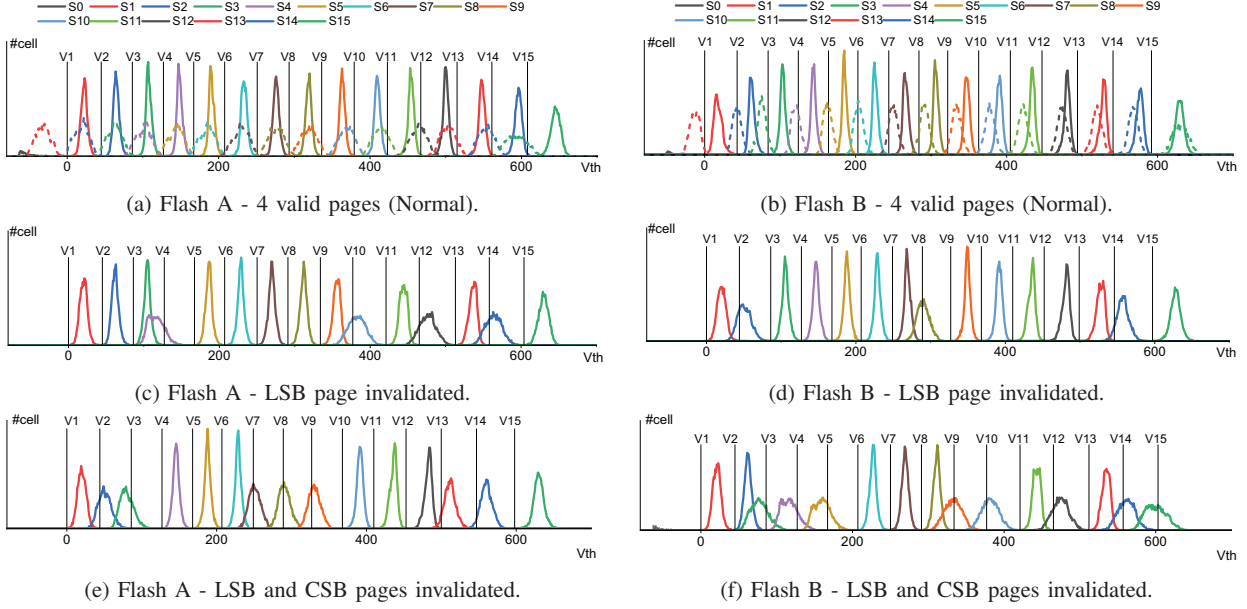


Fig. 10: The V_{th} distributions of the normal programming and NP programming. The dashed lines represent the V_{th} distribution after the first-step programming, while solid lines represent that after the second-step programming.

	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7	S_8	S_9	S_{10}	S_{11}	S_{12}	S_{13}	S_{14}	S_{15}
LSB	1	1	1	1	0	0	0	0	0	0	1	1	0	0	1	1
CSB	1	1	1	0	0	1	1	1	1	0	0	0	0	0	0	1
MSB	1	1	0	0	0	0	0	1	1	1	1	1	1	0	0	0
TSB	1	0	0	0	0	0	1	1	0	0	0	1	1	1	1	1

Diagram illustrating the bit positions V_1 through V_{15} and the corresponding bit values for the LSB, CSB, MSB, and TSB rows. The bits are grouped into four sets of four, each indicated by a blue arrow pointing to the corresponding group of bits in the rows below.

Fig. 11: Data mapping when the LSB page is invalid.

for other pages by mapping a voltage state to its left neighbor state. For the case with 2 or 3 pages being invalidated, there might be multiple consecutive voltage states with the same data bits on the valid pages. For example, when LSB and CSB pages are both invalid, the data bits in MSB and TSB are all the same from S_2 to S_5 . In this case, mapping S_3 to S_2 does not introduce reliability enhancement, as V_4 will not be applied for reading on the WL. Nevertheless, we still perform the mapping for the consideration of energy saving because fewer cells are charged during the second-step programming. The detailed energy saving will be presented in the evaluation section. In conclusion, for an arbitrary number of pages being invalidated, we will perform the voltage state mapping, either for reliability enhancement or energy saving.

D. Adaptive Data Allocation in SLC and QLC

The chief culprit of invalid programming is the existence of the time period between two programming steps, which is not removable because of the programming sequence constraint in 3D QLC flash. In this section, we aim to propose a dynamic data allocation between SLC and QLC regions to relieve the stress of invalid programming. Data allocation between two regions in current SSDs is performed based

on the data hotness, which allocates hot and cold data to SLC and QLC regions, respectively. We adopt the existing data hotness partitioning methods [26], [27], [31], [47] for hotness identification and further take the situation of invalid programming into consideration for data allocation. As an LPN could encounter invalid programming many times, we introduce a new Least-Recently Used (LRU) list in the SSD controller to record the pages that trigger invalid programming, termed *IPL* (invalid programming list). *IPL* takes both QLC invalid programming and SLC write pressure into consideration. If the invalid programming in the QLC region is increasing, more update requests dedicated to these invalid pages should be redirected to the SLC region, as they have a high probability of triggering invalid programming in the future. But if more pages are programmed into the SLC region, the SLC write pressure would be increased, thus leading to more data migration between the two regions. Therefore, the data allocation between SLC and QLC regions should be dynamically controlled.

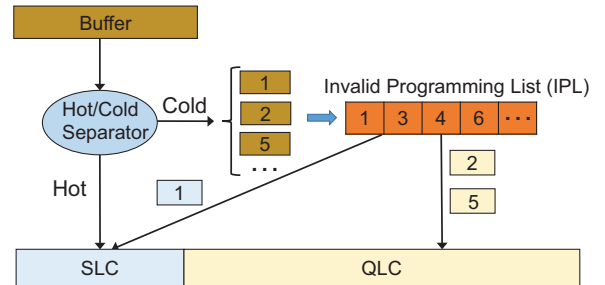


Fig. 12: The illustration of the adaptive data allocation.

Algorithm 1 Adaptive Data Allocation in SLC and QLC

Input: Page LPN_1 from buffer.

Output: Allocate LPN_1 to SLC Region or QLC Region.

```
1: Identify hotness of  $LPN_1$  via Hot/Cold Separator;  
2: if  $LPN_1$  is hot then  
3:   Write to SLC Region;  
4: else  
5:   if  $LPN_1$  is in IPL then  
6:     Write to SLC Region;  
7:     Move  $LPN_1$  to the LRU Tail;  
8:   else  
9:     Write to QLC Region;  
10:  end if  
11: end if
```

Algorithm 1 shows the data allocation between SLC and QLC regions, by considering the hotness and invalid programming. The cold data triggering invalid programming would be allocated to the SLC region. Figure 12 illustrates an example of adaptive data allocation. After the hot/cold separator assigns the writes to QLC, LPNs $LPN-1$, $LPN-2$ and $LPN-5$ are identified as cold data. In the traditional strategy, they will be written to the QLC region. For the consideration of invalid programming, we will check if they exist in IPL. $LPN-1$ is found in IPL and is therefore reassigned to the SLC region, while $LPN-2$ and $LPN-5$ are not in the IPL initially and are written to the QLC region as originally allocated.

Invalid Programming List Design: In the implementation of *IPL*, the list length is used as the indicator to dynamically throttle the number of update requests redirected from QLC to SLC region. The length of IPL directly impacts the tradeoff between the invalid programming severity and SLC write pressure. The severity of invalid programming is quantified through *invalid ratio*. We define *SLC write ratio*, as the ratio of writes in the SLC region to the total write-back pages, to quantify the SLC write pressure. The buffer write-back amount serves as a logical clock for monitoring the SLC write ratio and the invalid ratio. Within a certain write-back amount, the writes to the SLC region and the number of LPNs that undergo invalid programming are counted and recorded. Consequently, the current SLC write ratio and current invalid ratio can be calculated. By continuously monitoring the SLC write ratio and the invalid ratio, the length of IPL can be adaptively adjusted. The first priority is to control the SLC write ratio. Two thresholds are introduced to limit the lower and higher bounds of the SLC write ratio. When the current SLC write ratio exceeds the upper bound, the length of the IPL will be halved, and LPNs beyond the reduced length will be discarded. Conversely, when the current SLC write ratio falls below the lower threshold, the IPL length will double. For other situations where the SLC write ratio is within the normal range, the current invalid ratio is taken into consideration. If the current invalid ratio exceeds a certain threshold, the IPL length will also double.

TABLE I: The parameter settings.

Erase Latency	6.0ms
SLC Program Latency	0.1ms
QLC 1st-step Program Latency	1.4ms
QLC 2nd-step Program Latency	2.8ms
Pre-filling Ratio	0.30
Pre-invalid Ratio	0.15
GC Threshold	0.50

V. EVALUATION

A. Experiment Setup

The experiments are conducted on real 3D QLC flash chips as well as a simulator. Specifically, the effectiveness of the not-programming scheme is conducted on real chips and other experiments are conducted on the simulator. For the real flash experiments, two 3D QLC NAND flash models from two different vendors are evaluated. In Flash A, each block comprises 1536 pages, with 96 layers per block. Each layer contains 4 WLs, and each WL consists of four pages, with a page size of 16KB. The programming sequence of this chip follows the order as illustrated in Figure 2. In Flash B, a flash block has 128 layers, each of which contains 6 WLs. The first layer, the last layer, and the two intermediate layers of each block consist of TLC WLs. Thus each block has 3048 pages with the page size 16KB.

For the simulator, our experiments are conducted on *SimpleSSD*, which has been widely adopted in previous research [12], [19], [22], [24]. We implement a basic SLC-QLC hybrid SSD model on SimpleSSD. The buffer size in the model is set to 16MB. The hot/cold separator employed in this study is an LRU list [26], capable of accommodating 64 LPNs by default. The total capacity of the SSD is set to 1920GB, with the SLC region occupying 15% of the total capacity. The number of QLC blocks that could be written in parallel is set to 8. Each physical block contains 1536 pages and the page size is 16KB, which is the same as the first flash model we evaluated above. The settings for write and erase operations are presented in Table I, based on our real-device flash testing (the first four parameters) and the previous work [1], [9] (the remaining parameters).

We integrated the re-programming scheme, the not-programming scheme, and the proposed adaptive data allocator into the basic SLC-QLC hybrid SSD model. The initial length of the invalid programming list in the adaptive data allocator was set to 64, with a minimum value of 32 and a maximum value of 2048. The re-programming scheme was applied exclusively to WLs that contain only 1 valid page, while the not-programming scheme was applied to WLs containing 2 or 3 valid pages.

Finally, we selected workloads with diverse write characteristics from the Microsoft Cambridge Research [37], [44], [48]. These workloads were chosen to represent a wide range of writing patterns and behaviors, providing a comprehensive evaluation of the proposed design. The characteristics of the selected workloads are presented in Table II.

TABLE II: The characteristics of the selected workloads.

Workload	Write Ratio	Average Write Size (KB)
mds_0	88.1%	7.24
stg_0	84.8%	9.19
src2_2	69.7%	51.11
usr_0	59.6%	10.28
web_1	45.9%	9.22
prn_1	24.7%	11.66
proj_0	87.5%	40.92
prxy_0	96.9%	4.65
wdev_0	79.9%	8.20

B. Experiment Results

1) *Valid Page Threshold Determination*: *Valid page threshold* is introduced to decide whether re-programming should be performed. Specifically, re-programming is executed when the number of valid pages is less than or equal to the *valid page threshold*. When the valid page threshold is set to 0, it indicates that re-programming will not be conducted, serving as the baseline for comparison. This section first presents a study on the proper setting of *valid page threshold*.

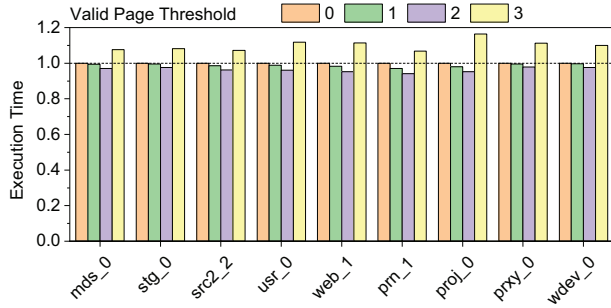


Fig. 13: The comparison of execution time.

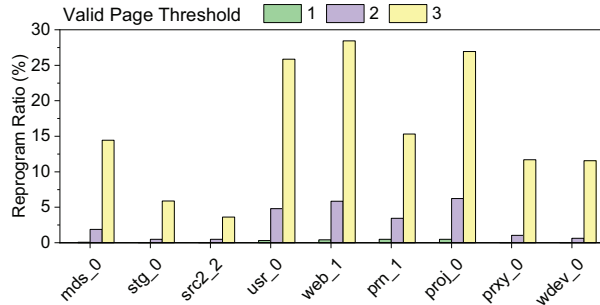


Fig. 14: The reprogramming ratio.

Figure 13 shows the comparison of the execution time. Execution time is the total time spent on data programming, including the first and second programming steps. As shown in Figure 13, when the *valid page threshold* is set to 1 or 2, there is a noticeable reduction in execution time, compared to the baseline method without re-programming. However, when it is set to 3, the execution time becomes higher than the baseline. This is consistent with our discussion in Section IV, as too many valid pages would introduce high re-programming overheads. On the other hand, the write latency has different behaviors from execution time. Only when the *valid page threshold* is set to 1, re-programming strategy could gain some

TABLE III: The parameter settings of adaptive data allocator.

t_1	The lower bound of SLC write ratio	0.15
t_2	The higher bound of SLC write ratio	0.70
t_3	The threshold of invalid ratio	0.07
MinLen	The minimum length of IPL	32
MaxLen	The maximum length of IPL	2048

benefits, resulting in a reduction of latency by up to 4.9%, with an average reduction of 3.1%. By setting it to 3, all workloads suffer from a significant increase in the write latency. By setting to 2, the write latency increases for some workloads, such as *usr_0* and *prn_1*. This is because increased execution time could increase the queuing delay of requests, which amplifies the write latency.

Figure 14 illustrates the ratio of re-programmed pages for different valid page thresholds. As the threshold value increases, the reprogramming ratio also increases. When the threshold is 1, the reprogramming ratio is trivial. While setting to larger values, there is a significant space wastage. Considering programming latency, and space utilization together, the most effective re-programming is achieved when there is only 1 valid page. Therefore, the *valid page threshold* is set to 1 for the evaluated chips in this work. We can tune the setting for other 3D flash models with different specifications.

2) *Invalid Ratio Reduction of Adaptive Data Allocator*: In order to compare the performance of the adaptive data allocator, we conducted tests on the following four schemes.

- **Baseline**: The baseline scheme that allocates data to SLC and QLC regions based on the data hotness.
- **ADA**: The adaptive data allocation proposed in Section IV-D. The relevant parameter settings of ADA are provided in Table III.
- **Infinite-IPL**: The strategy of setting the infinite length of IPL. Any page that triggers invalid programming in the QLC region will be recorded and then allocated to the SLC region once updated.
- **Ideal**: The ideal case in which we exploit with the prior knowledge of which pages frequently triggering invalid programming. These pages are directly written to the SLC region. This approach is implemented through two separate runs. The first run is used to collect invalid programming information, and the second run redirects QLC write requests for LPNs with multiple occurrences of invalid programming to the SLC region.

Figure 15 illustrates the invalid ratio of the four schemes. While the invalid ratio associated with ADA is not as low as that of *Ideal*, there is still a significant reduction compared to *Baseline*, with the invalid ratio reduced by 13.9% to 46.5%. It is important to note that the invalid ratio of *Ideal* is not theoretically as low. For example, according to Figure 5, over 80% of LPNs in workload *proj_0* experienced multiple occurrences of invalid programming, while *Ideal* only achieves a reduction of less than 40% in reduction of invalid ratio compared to *Baseline*. This is because data migration from SLC to QLC can still lead to invalid programming, as there will always be a time gap between the two programming steps.

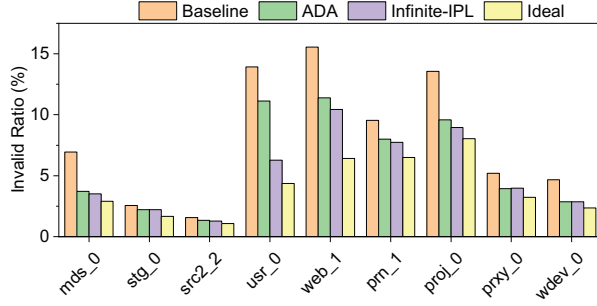


Fig. 15: The comparison of the invalid ratio.

After migrating some data to SLC, some QLC write operations that did not experience invalid programming may encounter invalid programming due to extended waiting time from fewer writes. The principle applies to the Infinite-IPL and ADA.

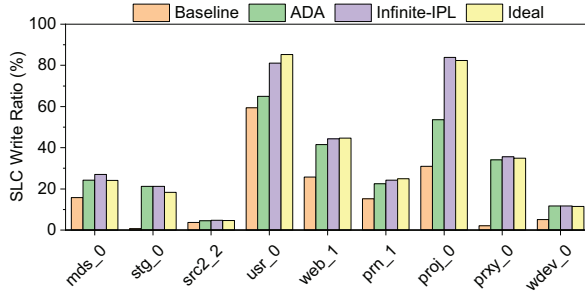


Fig. 16: The SLC write ratio of the four schemes.

Figure 16 illustrates the SLC write ratio of the four schemes. The results demonstrate a significant increase in SLC write ratio for both Infinite-IPL and Ideal, despite their lower invalid ratio. For trace *usr_0*, the SLC write ratio of Ideal reaches as high as 85.2%, indicating that SLC utilization can become critically high, leading to triggering GC operations, which can negatively impact performance. In contrast, our proposed ADA maintains the SLC write ratio within a relatively low range.

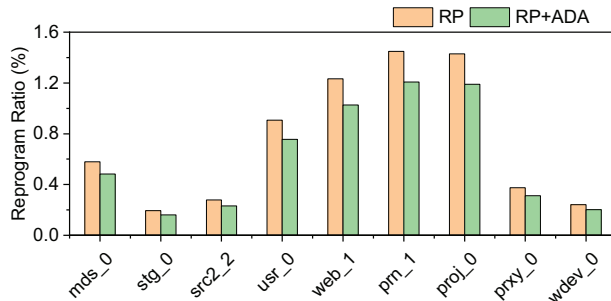


Fig. 17: The reprogram ratio of combining RP and ADA.

3) *Performance Improvement of RP*: As presented above, *valid page threshold* is set as 1 in the evaluation in RP strategy, which means a WL is only reprogrammed when the number of valid pages is 1 before the second programming step. Figure 17 shows the reprogramming ratio. Under this setting,

RP only introduces 0.74% reprogram operations, on average. With the combination of ADA, the average reprogramming ratio is 0.62%. Therefore, this strategy incurs little overhead for the performance.

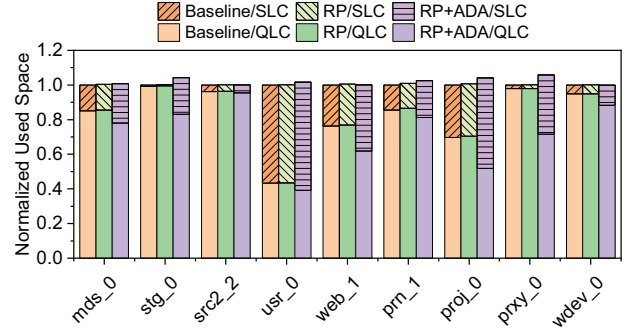


Fig. 18: The normalized used space.

Figure 18 further shows the normalized space usage, which is measured by the number of programmed pages. We separate the space usage between SLC and QLC regions. The baseline in the figure means data is allocated based on the hot/cold separator, by allocating hot and cold data to SLC and QLC regions, respectively. In baseline, the space used from two partitions varies among workloads because of their different access behaviors. The proposed RP strategy hardly changes the occupation of two regions because only a very small percentage of data is reprogrammed. Combining ADA, the pages that experienced invalid programming are allocated to SLC region, thus the space usage is increased. Nevertheless, the total used space by the two regions hardly changes. This means the proposed strategies induce little space overhead.

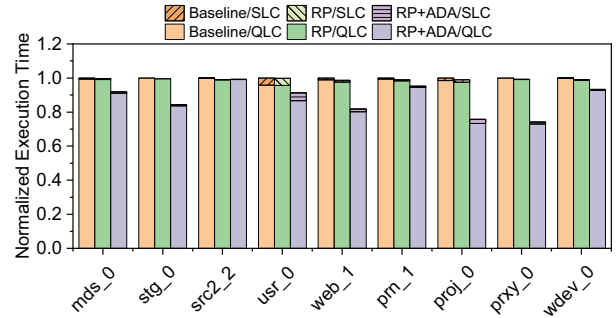


Fig. 19: The normalized execution time.

Figure 19 presents the normalized execution time, i.e., the time spent on data programming. As shown in the figure, even though the proposed strategies slightly increase the program count, the execution time is reduced by 13.51%, on average.

4) *Reliability Improvement in NP*: To perform a fair evaluation of the NP scheme, we first collect the number and locations of invalid pages occurring in several traces on a real 3D QLC chip. We observed that, approximately 200 pages in each block experience invalid programming, on average. As a block contains 384 WLs, if each partially-invalid WL has 1 page triggering invalid programming, about half of the WLs in a block will experience invalid programming.

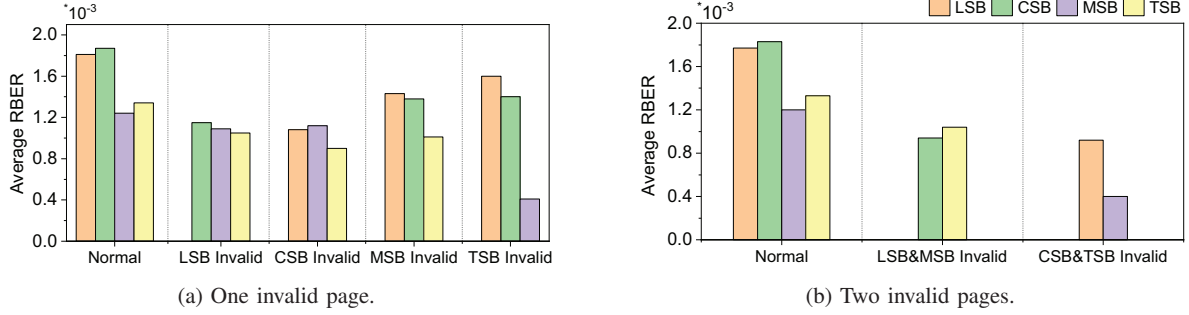


Fig. 20: The comparison of average RBER of the valid pages of partially-invalid WLs.

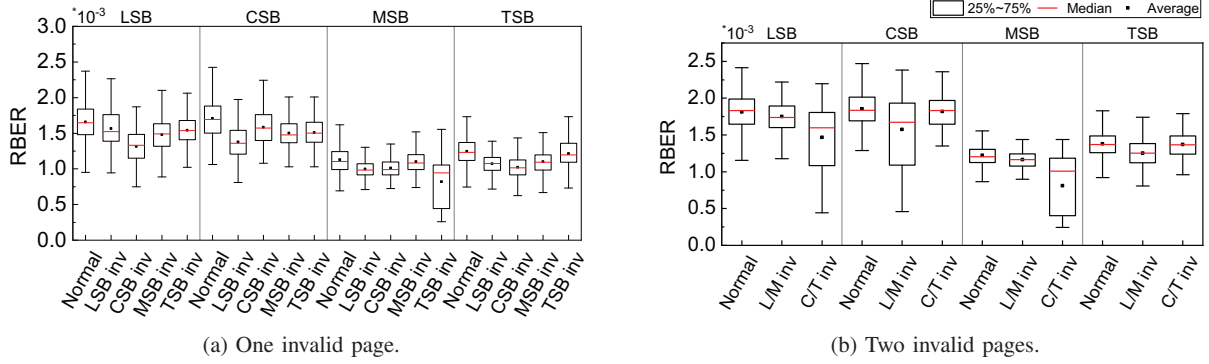


Fig. 21: The comparison of overall reliability, where normal programmed and NP-programmed WLs are all counted.

Therefore, we selected a trace with a near-average number of invalid pages and simulated the positions and quantities of WLs with occurrences of invalid programming on the QLC chip. We applied the not-programming scheme to these WLs (192 WLs in this experiment), which involves modifying the programming data in the second-step programming, thereby improving the reliability of valid pages. Other WLs in the blocks are normally programmed.

We first present the reduction in the raw bit error rate (RBER) of the partially-invalid WLs with the proposed NP scheme. Figure 20 compares the RBER of the valid pages of partially-invalid WLs. Normal in the figure represents the baseline method, normally programming a flash block, and XSB Invalid represents the NP scheme with the XSB page invalidated (of some WLs in a block) before the second-step programming. Taking LSB Invalid in Figure 20a as an example, not-programming LSB pages can enlarge the noise margins between voltage states corresponding to the other three pages, thus reducing their RBER. Compared to Normal, LSB Invalid reduces the RBER of CSB, MSB, and TSB pages by 38.50%, 12.09%, and 21.64%, respectively. Figure 20a shows that, with 1 page invalidated, the NP scheme can reduce the RBER of four pages by 40.33%, 38.50%, 66.94%, and 32.84%, in the best case. With 2 pages invalidated, the reduction is more significant, as shown in Figure 20b. We tested all cases where two pages were invalidated, and only partial results were selected and presented in this figure due to the page restriction. The experimental results for all combinations indicate that using the NP scheme leads to

improved reliability of valid pages. Inside a flash block, not all WLs endure invalid programming and some WLs have all four pages staying valid during programming. We also studied the impacts of NP on these WLs. We found that there is little difference in the average RBER for WLs without invalid pages between normal programming and using the NP scheme.

To gain a better understanding of the reliability impacts, Figure 21 compares the overall reliability, which includes the RBER data from both normally programmed and NP-programmed WLs of each block. The y-axis represents the RBER distribution of different pages within a block across all WLs. With the NP scheme, the RBER range is lower than that of the normal program. The results indicate that the NP scheme can improve the overall reliability by effectively reducing the RBER of the valid pages. The greater the number of invalid pages, the more significant the improvement in the reliability of other valid pages. This is attributed to the fact that each invalid page contributes to the reliability enhancement of a specific state in the valid pages. As the number of pages in a WL increases, like cells with higher density than QLC, the probability of triggering invalid programming increases, as well as the number of invalid pages. More states in the valid pages would benefit from such reliability enhancement, leading to a more pronounced overall improvement in reliability.

Flash memory is equipped with a refresh mechanism, by periodically reading, correcting, and programming data to new blocks for reducing accumulated errors over retention time. Because a 3D QLC block has both NP-programmed WLs (with RBER reduction) and normally programmed WLs (without

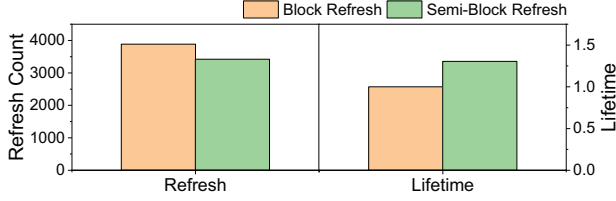


Fig. 22: The comparison of refresh count and lifetime.

RBER reduction), we propose to separately refresh these two types of WLs with different frequencies, called *Semi-Block Refresh*. The refresh of a block is performed through two steps. First, normally programmed data are migrated when the retention time reaches the normal refresh period. Second, NP-programmed data are migrated after a longer refresh period because of the enhanced reliability. Upon completion of the second data migration, the entire block is erased. Figure 22 shows the reduction in refresh count and prolonging of lifetime. With *Semi-Block Refresh*, the block refresh count is reduced by 11.89% and the lifetime is extended by 30.43%.

5) *Energy Saving of NP*: As the NP scheme skips the fine programming of some states, the energy consumption of the second-step programming can be reduced. To provide an effective energy consumption model for 3D QLC, we derive the existing work [35], [36] to perform some simple modeling of programming energy consumption in the second step.

QLC adopts ISPP (Incremental Step Pulse Programming) to incrementally increase sustained sub-programming operations for programming all cells to the target voltage [43]. The energy consumption of programming (E_{pgm}) is determined by the number of sub-programming cycles n and the energy of sub-programming operations (E_{subp}), which can be represented as $E_{pgm} = \sum_0^n E_{subp(j)}$. The energy consumption of the j th sub-programming operation mainly comes from two aspects. The first is the energy consumption of common operations (E_{common}) such as wordline selection, programming verification, inhibiting programming, and so on, which is constant. The second is the energy consumption of injecting electrons into each cell. This part (E_p) can be calculated by the formula $E_p = k * \Delta V_{th}$, where k is a constant affected by sub-programming time and sub-programming current and ΔV_{th} is the voltage change value of a cell during the sub programming operation. The energy consumption of each sub-programming operation is $E_{subp(j)} = E_{common} + \sum E_p$. Considering that the cells will no longer undergo programming after reaching the target voltage, E_{pgm} could be further simplified as $n * E_{common} + n_p * k * (V_s - V_f)$, where n_p is the number of cells requiring programming during fine programming, where V_s and V_f are the target voltage of fine programming and the voltage after coarse programming, respectively.

For normally programmed cells, their $V_s - V_f$ values are essentially fixed, which means that E_{pgm} mainly depends on the number of cells undergoing normal programming. Suppose

TABLE IV: The number of not-programming states N_{np} and the energy consumption saving ratio (ESR).

Invalid Page	N_{np}	ESR(%)	Invalid Page	N_{np}	ESR(%)
LSB	4	12.5	LSB&MSB	7	21.875
CSB	4	12.5	LSB&TSB	8	25
MSB	3	9.375	CSB&MSB	7	21.875
TSB	4	12.5	CSB&TSB	8	25
LSB&CSB	8	25	MSB&TSB	7	21.875

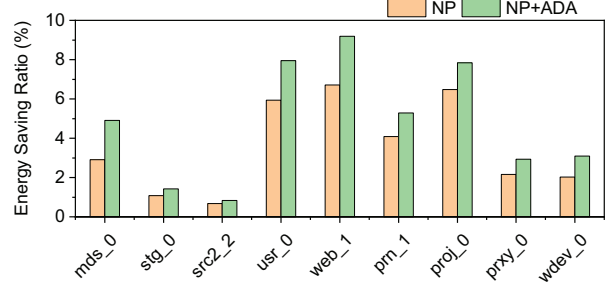


Fig. 23: The energy saving ratio of different workloads.

that E_{pgm} of all cells requiring programming during fine programming is E_{normal} and E_{pgm} of all cells not requiring programming during fine programming is E_{notp} . When the data is uniformly distributed, the number of cells undergoing normal programming is determined by the number of states that do not require fine-grained programming (N_{np}) in the NP scheme. Thus the energy consumption saving ratio (ESR) can be calculated by $\frac{N_{np}}{16} * (1 - \frac{E_{notp}}{E_{normal}})$. Through the calculation, we can get the energy saving of NP scheme, shown in Table IV. With 1 page invalidated, we can have an average energy reduction of 11.7%, while 2 invalid pages result in an average energy reduction of 23.4%.

Based on Table IV and the evaluated distribution of invalid pages for each workload, we can obtain the energy saving for applications as shown in Figure 23. The NP scheme achieved energy reduction of fine programming, with an average reduction of 3.5% and a maximum reduction of 6.6%. By combining NP with ADA, energy reduction is larger, with an average reduction of 4.8% and a maximum reduction of 9.1%.

VI. RELATED WORKS

Flash programming. Choi et al. [7] proposed to merge the voltage states that represent the same data bits when one or several pages become invalid. This method requires re-programming the valid data on the same WL. While it works on TLC flash, it leads to data corruption with severe state overlapping on 3D QLC flash memory, as verified by our evaluations on multiple 3D QLC flash models. Several works proposed re-programming SLC by programming each cell multiple times after invalidating previous values in the same cell [3], [6]. Gao et al. [11] proposed to re-program 3D TLC flash by first programming some of the voltage states. Some subpage programming strategies are proposed to program parts of a physical WL by operating data in a fine granularity [23], [25]. They are only applicable for low-density flash memories that have a large noise margin between voltage states and no restriction on the program sequence.

High-density flash coding. Lv et al. [32] present that different gray codes vary in performance and reliability, based on which they propose to use the specific gray code for different purposes in 3D QLC flash. To further increase the capacity of flash-based storage, write-once memory (WOM) code [18], [33], [34], [49] is adopted to increase the logical writes before an erase. However, the WOM code requires ECC to provide stronger error correction capability.

Hybrid-flash data management. Yang et al. [50] propose to dynamically turn TLC to SLC blocks for exploiting the high performance of SLC blocks, when the storage space is sufficient. Yoo et al. [51] adopt a reinforcement learning method to find the optimal SLC cache parameter in hybrid SSDs, by considering the workload properties and the internal status of SSD. Shi et al. [41] propose to schedule write requests based on the velocity and tune the capacity of hybrid SSD with the consideration of garbage collection.

VII. CONCLUSIONS

3D QLC flash memory adopts a two-step programming strategy in WL granularity. This strategy with a predefined programming order introduces an invalid programming issue for 3D QLC flash. To deal with this problem, this paper proposes to exploit the invalid data on partially-invalid WLs to enhance reliability and performance for valid data. Three strategies are proposed to revise the programming mechanism of 3D QLC NAND flash storage. The real-device evaluations show the proposed strategies could improve data reliability and performance without introducing extra overheads. Flash cells with more bits, such as PLC with 5 bits per cell, would suffer from even severe invalid programming issues because the probability of a WL becoming partially-invalid is higher, which indicates a larger design space of high-efficiency programming.

ACKNOWLEDGEMENT

We sincerely thank the shepherd and anonymous reviewers for their constructive feedback. This work is supported in part by the National Key Research and Development Program of China (Grant No. 2023YFB4502703), the National Natural Science Foundation of China under Grant No. 62202396, 62332021, and 62102219, and the Natural Science Foundation of Xiamen under Grant No. 3502Z20227023. The corresponding author is Congming Gao.

REFERENCES

- [1] J. Bae, J. Park, Y. Jun, and E. Seo, "Dedup-for-speed: storing duplications in fast programming mode for enhanced read performance," in *Proceedings of the 15th ACM International Conference on Systems and Storage*, 2022, pp. 128–139.
- [2] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1666–1704, 2017.
- [3] T.-Y. Chen, Y.-H. Chang, Y.-H. Kuan, and Y.-M. Chang, "Virtualgc: Enabling erase-free garbage collection to upgrade the performance of rewritable slc nand flash memory," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017, pp. 1–6.
- [4] W. Cho, J. Jung, J. Kim, J. Ham, S. Lee, Y. Noh, D. Kim, W. Lee, K. Cho, K. Kim *et al.*, "A 1-tb, 4b/cell, 176-stacked-wl 3d-nand flash memory with improved read latency and a 14.8 gb/mm2 density," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 134–135.
- [5] W. Cho, J. Jung, J. Kim, J. Ham, S. Lee, Y. Noh, D. Kim, W. Lee, K. Cho, K. Kim, H. Lee, S. Chai, E. Jo, H. Cho, J.-S. Kim, C. Kwon, C. Park, H. Nam, H. Won, T. Kim, K. Park, S. Oh, J. Ban, J. Park, J. Shin, T. Shin, J. Jang, J. Mun, J. Choi, H. Choi, S.-W. Choi, W. Park, D. Yoon, M. Kim, J. Lim, C. An, H. Shirr, H. Oh, H. Park, S. Shim, H. Huh, H. Choi, S. Lee, J. Sim, K. Gwon, J. Kim, W. Jeong, J. Choi, and K.-W. Jin, "A 1-tb, 4b/cell, 176-stacked-wl 3d-nand flash memory with improved read latency and a 14.8gb/mm2 density," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 134–135.
- [6] W. Choi, M. Arjomand, M. Jung, and M. Kandemir, "Exploiting data longevity for enhancing the lifetime of flash-based storage class memory," in *Proceedings of the Measurement and Analysis of Computing Systems*. ACM, 2017.
- [7] W. Choi, M. Jung, and M. Kandemir, "Invalid data-aware coding to enhance the read performance of high-density flash memories," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 482–493.
- [8] H. Dang, X. Yao, Z. Wan, and Q. Li, "A study of invalid programming in 3d qlc nand flash memories," in *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*, 2023, pp. 73–79.
- [9] J. Do, I. L. Picoli, D. Lomet, and P. Bonnet, "Better database cost/performance via batched i/o on programmable ssd," *The VLDB Journal*, vol. 30, pp. 403–424, 2021.
- [10] M. Fukuchi, S. Suzuki, K. Maeda, C. Matsui, and K. Takeuchi, "Ber evaluation system considering device characteristics of tlc and qlc nand flash memories in hybrid ssds with real storage workloads," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–4.
- [11] C. Gao, M. Ye, Q. Li, C. J. Xue, Y. Zhang, L. Shi, and J. Yang, "Constructing large, durable and fast ssd system via reprogramming 3d tlc flash memory," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 493–505.
- [12] D. Gouk, M. Kwon, J. Zhang, S. Koh, W. Choi, N. S. Kim, M. Kandemir, and M. Jung, "Amber: Enabling precise full-system simulation with detailed modeling of all ssd resources," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 469–481.
- [13] F. Gray, "Pulse code communication," *United States Patent Number 2632058*, 1953.
- [14] K. Han, H. Kim, and D. Shin, "Wal-ssd: Address remapping-based write-ahead-logging solid-state disks," *IEEE Transactions on Computers*, vol. 69, no. 2, pp. 260–273, 2019.
- [15] P. Huang, P. Subedi, X. He, S. He, and K. Zhou, "Flexecc: Partially relaxing ecc of mlc ssd for better cache performance," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 489–500.
- [16] H. Huh, W. Cho, J. Lee, Y. Noh, Y. Park, S. Ok, J. Kim, K. Cho, H. Lee, G. Kim, K. Park, K. Kim, H. Lee, S. Chai, C. Kwon, H. Cho, C. Jeong, Y. Yang, J. Goo, J. Park, J. Lee, H. Kirr, K. Jo, C. Park, H. Nam, H. Song, S. Lee, W. Jeong, K.-O. Ahn, and T.-S. Jung, "13.2 a 1tb 4b/cell 96-stacked-wl 3d nand flash memory with 30mb/s program throughput using peripheral circuit under memory cell array technique," in *2020 IEEE International Solid- State Circuits Conference - (ISSCC)*, 2020, pp. 220–221.
- [17] S. Jaffer, K. Mahdavian, and B. Schroeder, "Rethinking wom codes to enhance the lifetime in new ssd generations," in *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*, 2020.
- [18] S. Jaffer, K. Mahdavian, and B. Schroeder, "Improving the reliability of next generation ssds using wom-v codes," in *20th USENIX Conference on File and Storage Technologies (FAST 22)*, 2022, pp. 117–132.
- [19] M. Kang, W. Lee, J. Kim, and S. Kim, "Pr-ssd: Maximizing partial read potential by exploiting compression and channel-level parallelism," *IEEE Transactions on Computers*, vol. 72, no. 3, pp. 772–785, 2022.
- [20] B. Kim and M. Kim, "Lazyrs: Improving the performance and reliability of high-capacity tlc/qc flash-based storage systems using lazy reprogramming," *Electronics*, vol. 12, no. 4, p. 843, 2023.
- [21] B. S. Kim, J. Choi, and S. L. Min, "Design tradeoffs for ssd reliability," in *17th USENIX Conference on File and Storage Technologies (FAST 19)*, 2019, pp. 281–294.

- [22] J. Kim, S. Kang, Y. Park, and J. Kim, "Networked ssd: Flash memory interconnection network for high-bandwidth ssd," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 388–403.
- [23] J.-H. Kim, S.-H. Kim, and J.-S. Kim, "Subpage programming for extending the lifetime of nand flash memory," in *Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 2015.
- [24] J. Kim, M. Kang, Y. Han, Y.-G. Kim, and L.-S. Kim, "Optimstore: In-storage optimization of large scale dnns with on-die processing," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 611–623.
- [25] M. Kim, J. Lee, S. Lee, J. Park, and J. Kim, "Improving performance and lifetime of large-page nand storages using erase-free subpage programming," in *Design Automation Conference*. IEEE, 2017.
- [26] J. Lee and J.-S. Kim, "An empirical study of hot/cold data separation policies in solid state drives (ssds)," in *Proceedings of the 6th International Systems and Storage Conference*, 2013, pp. 1–6.
- [27] B. Li, C. Deng, J. Yang, D. Lilja, B. Yuan, and D. Du, "Haml-ssd: A hardware accelerated hotness-aware machine learning based ssd management," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–8.
- [28] Q. Li, M. Ye, Y. Cui, L. Shi, X. Li, T.-W. Kuo, and C. J. Xue, "Shaving retries with sentinels for fast read over high-density 3d flash," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 483–495.
- [29] S. Li, L. Luo, Y. Lv, and L. Shi, "Latency aware page migration for read performance optimization on hybrid ssds," in *2022 IEEE 11th Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. IEEE, 2022, pp. 33–38.
- [30] L. Long, J. Huang, C. Gao, D. Liu, R. Liu, and Y. Jiang, "Adar: Application-specific data allocation and reprogramming optimization for 3d tlc flash memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
- [31] Y. Luo, Y. Cai, S. Ghose, J. Choi, and O. Mutlu, "Warm: Improving nand flash memory lifetime with write-hotness aware retention management," in *2015 31st Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2015, pp. 1–14.
- [32] Y. Lv, L. Shi, Q. Li, C. Gao, Y. Song, L. Luo, and Y. Zhang, "Mgc: Multiple-gray-code for 3d nand flash based high-density ssds," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 122–136.
- [33] F. Margaglia and A. Brinkmann, "Improving mlc flash performance and endurance with extended p/e cycles," in *Symposium on Mass Storage Systems and Technologies*. IEEE, 2015.
- [34] F. Margaglia, G. Yadgar, E. Yaakobi, Y. Li, A. Schuster, and A. Brinkmann, "The devil is in the details: Implementing flash page reuse with wom codes," in *Conference on File and Storage Technologies*, 2016.
- [35] V. Mohan, T. Bunker, L. Grupp, S. Gurumurthi, M. R. Stan, and S. Swanson, "Modeling power consumption of nand flash memories using flashpower," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 7, pp. 1031–1044, 2013.
- [36] V. Mohan, S. Gurumurthi, and M. R. Stan, "Flashpower: A detailed power model for nand flash memory," in *2010 Design, Automation Test in Europe Conference Exhibition (DATE 2010)*, 2010, pp. 502–507.
- [37] D. Narayanan, A. Donnelly, and A. Rowstron, "Write off-loading: Practical power management for enterprise storage," *ACM Transactions on Storage (TOS)*, vol. 4, no. 3, pp. 1–23, 2008.
- [38] D. Nicolas Bailon, J.-P. Thiers, and J. Freudenberger, "Error correction for tlc and qlc nand flash memories using cell-wise encoding," *Electronics*, vol. 11, no. 10, p. 1585, 2022.
- [39] S. Pang, Y. Deng, Z. Wu, G. Zhang, J. Li, and X. Qin, "Fspda: A full sequence program data allocation scheme for boosting 3d nand flash read performance," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2023.
- [40] J. Park, M. Kim, M. Chun, L. Orosa, J. Kim, and O. Mutlu, "Reducing solid-state drive read latency by optimizing read-retry," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 702–716.
- [41] L. Shi, L. Luo, Y. Lv, S. Li, C. Li, and E. H.-M. Sha, "Understanding and optimizing hybrid ssd with high-density and low-cost flash memory," in *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 2021, pp. 236–243.
- [42] N. Shibata, K. Kanda, T. Shimizu, J. Nakai, O. Nagao, N. Kobayashi, M. Miakashi, Y. Nagadomi, T. Nakano, T. Kawabe *et al.*, "A 1.33-tb 4-bit/cell 3-d flash memory on a 96-word-line-layer technology," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, pp. 178–188, 2019.
- [43] K.-D. Suh, B.-H. Suh, Y.-H. Lim, J.-K. Kim, Y.-J. Choi, Y.-N. Koh, S.-S. Lee, S.-C. Kwon, B.-S. Choi, J.-S. Yum, J.-H. Choi, J.-R. Kim, and H.-K. Lim, "A 3.3 v 32 mb nand flash memory with incremental step pulse programming scheme," *IEEE Journal of Solid-State Circuits*, vol. 30, no. 11, pp. 1149–1156, 1995.
- [44] J. Wan, W. Wu, L. Zhan, Q. Yang, X. Qu, and C. Xie, "Deft-cache: A cost-effective and highly reliable ssd cache for raid storage," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2017, pp. 102–111.
- [45] D. Wei, H. Feng, M. Liu, Y. Song, Z. Piao, C. Hu, and L. Qiao, "Edge word-line reliability problem in 3-d nand flash memory: Observations, analysis, and solutions," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [46] C. Xiao, S. Qiu, and D. Xu, "Pasm: Parallelism aware space management strategy for hybrid ssd towards in-storage dnn training acceleration," *Journal of Systems Architecture*, vol. 128, p. 102565, 2022.
- [47] W. Xie and Y. Chen, "An adaptive separation-aware fitl for improving the efficiency of garbage collection in ssds," in *2014 14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. IEEE, 2014, pp. 552–553.
- [48] G. Yadgar, M. Gabel, S. Jaffer, and B. Schroeder, "Ssd-based workload characteristics and their performance implications," *ACM Transactions on Storage (TOS)*, vol. 17, no. 1, pp. 1–26, 2021.
- [49] G. Yadgar, E. Yaakobi, and A. Schuster, "Write once, get 50% free: Saving ssd erase costs using wom codes," in *Conference on File and Storage Technologies*, 2015.
- [50] M.-C. Yang, Y.-H. Chang, C.-W. Tsao, and C.-Y. Liu, "Utilization-aware self-tuning design for tlc flash storage devices," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, no. 10, pp. 3132–3144, 2016.
- [51] S. Yoo and D. Shin, "Reinforcement Learning-Based SLC cache technique for enhancing SSD write performance," in *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*. USENIX Association, 2020.
- [52] K. Zhao, W. Zhao, H. Sun, X. Zhang, N. Zheng, and T. Zhang, "\$ldpc - in - ssd\$: Making advanced error correction codes work effectively in solid state drives," in *11th USENIX Conference on File and Storage Technologies (FAST 13)*, 2013, pp. 243–256.
- [53] Y. Zhou, Q. Wu, F. Wu, H. Jiang, J. Zhou, and C. Xie, "Remap-ssd: Safely and efficiently exploiting ssd address remapping to eliminate duplicate writes," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 187–202.